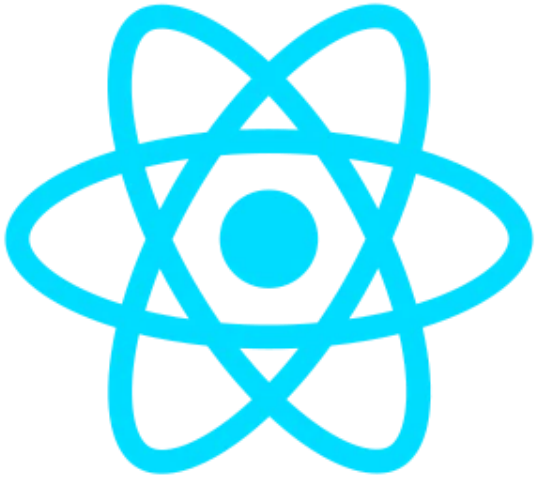
Toegepaste Informatica

UCLL HOGESCHOOL

Full-stack software development

Front-end introduction: React

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe



React JS

“A Javascript library for building user interfaces”

What is React?

Declarative views: Focus on designing UI, React handles rendering and updating.

Component based: Write pure JS/TS components with state and combine them to create complex UI's.

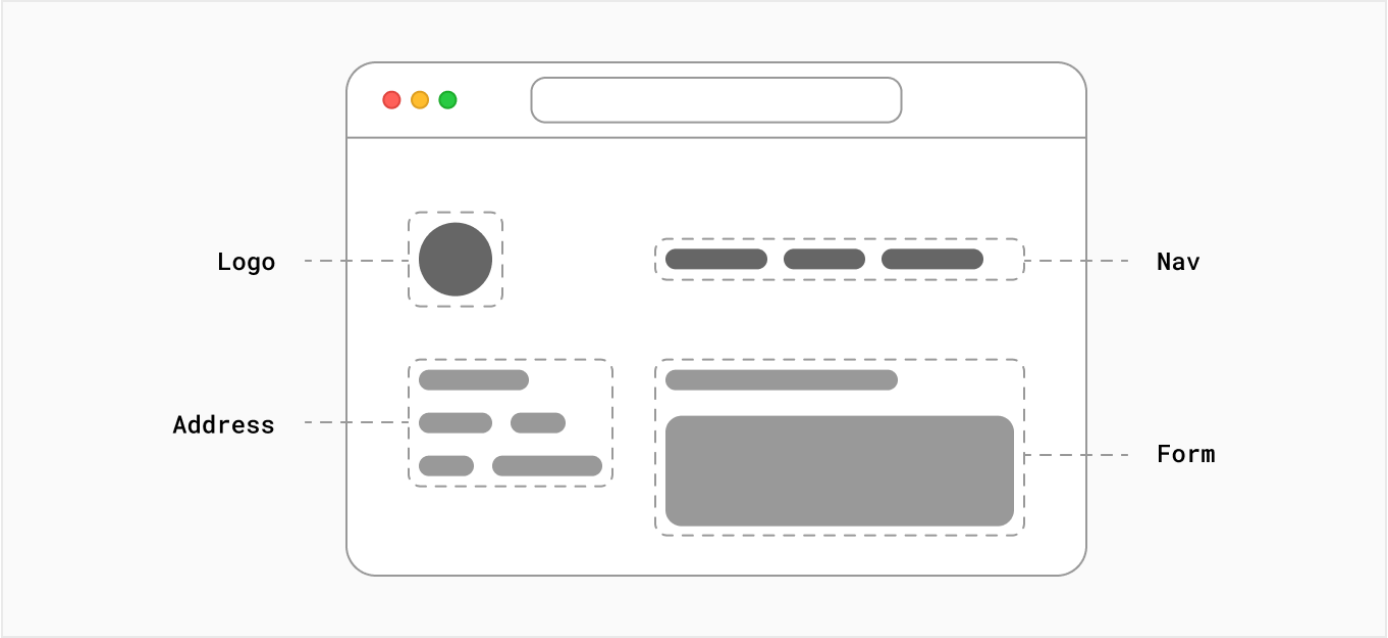
Built and used by Meta (Facebook / Instagram / WhatsApp).

Building Blocks of a Web Application

- **User Interface**
  - How users will interact with your application.
- **Routing**
  - How users navigate between different parts of your application.
- **Data Fetching**
  - Where your data lives and how to get it.
- **Rendering**
  - When and where you render static or dynamic content.

React

- A JavaScript **library** for building interactive **user interfaces**
  - User interfaces: **elements** that users **see** and **interact** with on-screen
  - Library: **helpful functions** to build UI

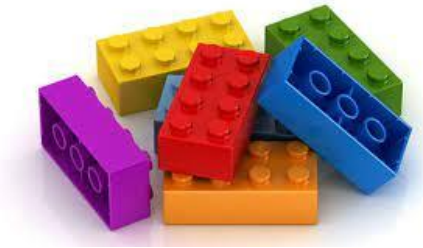


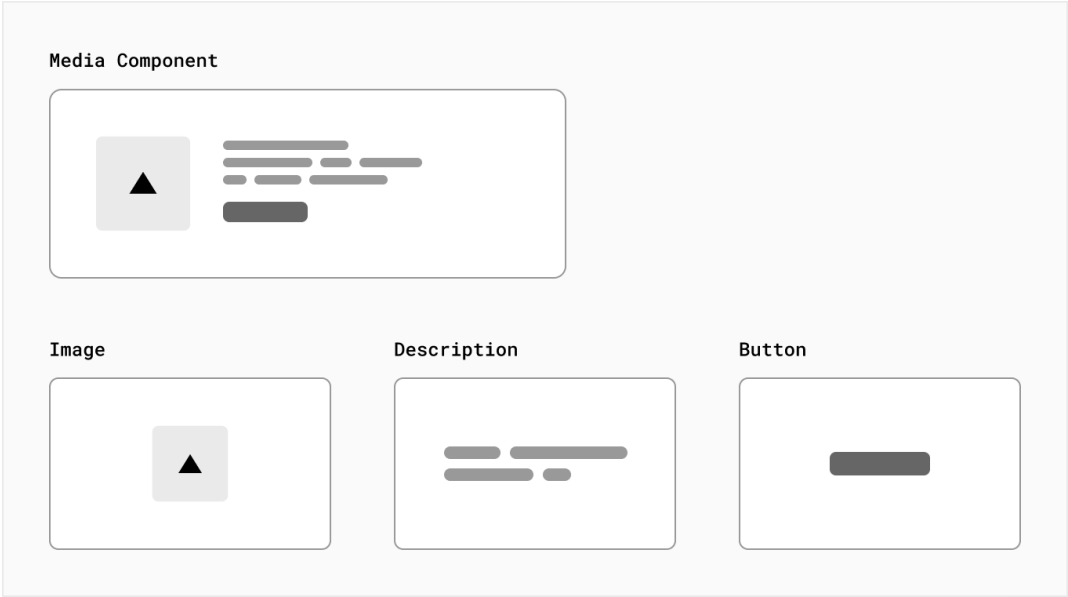
React

- Components
- Props
- Rendering
- Composition
- Lifecycle
- State
- Side effects

Components

- A User Interface can be broken down into smaller building blocks = components
  - components are like LEGO bricks
- Important benefit: modularity
  - more maintainable code
  - easily add, update and delete components





Components are functions

- Plain JavaScript Functions: A component in React is essentially a JavaScript function.
- Is used like an HTML tag.

```
type Props = {
    name: string
}

const Greeting: React.FC<Props> = ({name}: Props) => {

        return <h1>Hello, {name}!</h1>

}

<Greeting name="Elke Steegmans" />
```

Components have properties

- Input for components = Props
  - The component function can receive one argument: an object containing “properties” or props.
  - This is the data a component receives to determine how it should render.

Components are rendered

- Rendering is the process where React converts your component code into the final HTML that is shown in the browser.
- When? React “renders” a component:
  - The first time it appears on the screen.
  - Every time the component's props or internal state changes.
- How? React “executes” the component (function), its return value determines what will be rendered.

Components return value: JSX

- Describes the UI: The return value of a component is JSX (JavaScript XML).
- HTML in JavaScript: JSX is a syntax extension that lets you write HTML-like code directly in your JavaScript.
- The returned JSX is a “blueprint” that tells React what the UI should look like and which HTML elements to generate.

```
const name = 'Elke Steegmans'
const element = <h1>Hello, {name}!</h1>
```

Conditional Rendering

```tsx
type Props = {
    name: string
    loggedIn: boolean
}

const Header: React.FC<Props> = ({loggedIn}: Props) => {
    if (loggedIn) {
        return <div><p>Hello {name}</p></div>
    }
    return <div><p>Hello Guest!</p></div>
}
```

```tsx
type Props = {
    name: string
    loggedIn: boolean
}

const Header: React.FC<Props> = ({loggedIn}: Props) => {
    return (
        <div>
            loggedIn && <p>Hello {name}</p>
            !loggedIn && <p>Hello Guest!</p>
        </div>
    )
}
```

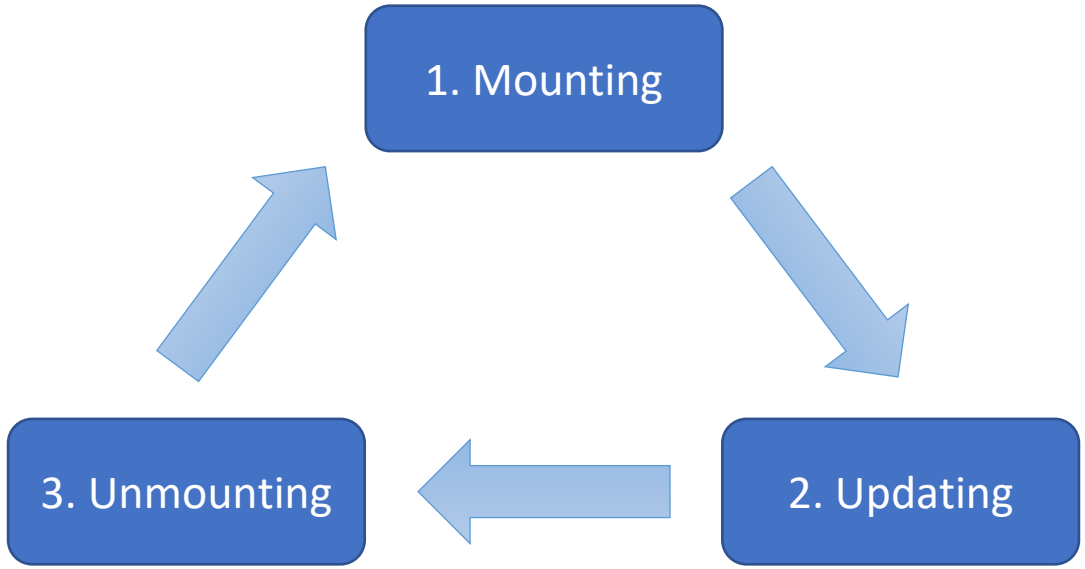
OR an inline IF with logical && operator

Composition of Components

- Use components like HTML tags.
- The return value of a component can include other components.

```
const Header: React.FC = () => {
    return (
        <div>
            <p>Welcome</p>
            <Greeting />
        </div>
    )
}
```

Lifecycle of a React component



Lifecycle: mount phase

- The component is created and added to the **DOM**.
- Component is rendered: the **return value** of the component function.
- The return value is a **single root HTML node**.
  - If you have multiple HTML nodes on the same level, you can use **fragments** to return a single node:

```
<>
  <div>…</div>
  <div>…</div>
</>
```

Lifecycle: updating phase

- Occurs when the component’s state or props change.
- Current state/props are compared with previous values and component is updated.
  - Render function is called again.
  - Goal: component displays latest version of itself.
- This phase repeats until the component is unmounted.

Lifecycle: unmounting phase

- Final phase of a component where it is **destroyed** and **removed** from the DOM.
- **Cleanup** of the component**:**
  - Event listeners
  - Timers
  - Cancelling external requests
  - …

Components have State

- Store information needed for a next re-render.
- Stored in a special state variable.
  - Information not stored in a state variable will be lost in the next re-render.
- When state itself is changed, React will re-render the component.

useState

- Allows you to add a state variable to the component
- Components often need to change what's on the screen because of an interaction and need to "remember" things: the current input value, the current image, the shopping cart.
- In React, this kind of component-specific memory is called ***state***.

useState

- Provides two things:
  - A **state variable** to retain the data between renders
  - A **state setter function** to update the variable and trigger React to render the component again
  - https://react.dev/learn/state-a-components-memory

```
const [name, setName] = useState<string | null>("");
```

Binding state to an input field

“Binding” = connecting the value field to the state variable:

```
const LoginForm = () => {
    const [name, setName] = useState('');

    return (
        <form onSubmit={handleSubmit}>
            <input id="nameInput" type="text" value={name} />
        </form>
    );
}
```

Modifying state

On every change in the input field, call the state setter function:

```
const LoginForm = () => {
    const [name, setName] = useState('');

    return (
        <form onSubmit={handleSubmit}>
            <input id="nameInput" type="text" value={name} onChange={(event) => setName(event.target.value)} />
        </form>
    );
}
```

Example component

Lecturers

LecturersClientPage component

Firstname	Lastname	E-mail	Expertise
Johan	Pieck	johan.pieck@ucll.be	Full-stack development, Front-end development
Elke	Steegmans	elke.steegmans@ucll.be	Software Engineering, Back-end Development
Greetje	Jongen	greetje.jongen@ucll.be	Full-stack development, Back-end Development

LecturersOverviewTable component

Courses taught by Elke:

Name	Description	Phase	Credits
Full-stack development	Learn how to build a full stack web application.	2	6
Software Engineering	Learn how to build and deploy a software application.	2	6

CoursesOverviewTable component

```tsx
import React from "react";
import { Lecturer } from "@types";

type Props = {
    lecturers: Array<Lecturer>;
    selectLecturer: (lecturer: Lecturer) => void;
};

const LecturerOverviewTable: React.FC<Props> = ({
    lecturers,
    selectLecturer,
}: Props) => {
    return (
        <>
            {lecturers && (
                <table className="table table-hover">
                    <thead>
                        <tr>
                            <th scope="col">Firstname</th>
                            <th scope="col">Lastname</th>
                            <th scope="col">E-mail</th>
                            <th scope="col">Expertise</th>
                        </tr>
                    </thead>
                    <tbody>
                        {lecturers.map((lecturer, index) => (
                            <tr
                                key={index}
                                onClick={() => selectLecturer(lecturer)}
                                role="button">
                                <td>{lecturer.user.firstName}</td>
                                <td>{lecturer.user.lastName}</td>
                                <td>{lecturer.user.email}</td>
                                <td>{lecturer.expertise}</td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            )}
        </>
    );
};

export default LecturerOverviewTable;
```

Callback function that parent components can pass as a prop to this component.

Destructure props in local variables.

Conditional rendering: only render table if lecturers is not empty.

Every element of lecturers is mapped to a table row

When clicking in a row, notify parent components of this event (and which lecturer is being clicked) by calling the callback function.

```tsx
type Props = {
    lecturer: Lecturer;
};

const CourseOverviewTable: React.FC<Props> = ({ lecturer }: Props) => {
    return (
        <>
            {lecturer.courses && (
                <table className="table table-hover">
                    <thead>
                        <tr>
                            <th scope="col">Name</th>
                            <th scope="col">Description</th>
                            <th scope="col">Phase</th>
                            <th scope="col">Credits</th>
                            <th scope="col"></th>
                        </tr>
                    </thead>
                    <tbody>
                        {lecturer.courses.map((course, index) => (
                            <tr key={index}>
                                <td>{course.name}</td>
                                <td>{course.description}</td>
                                <td>{course.phase}</td>
                                <td>{course.credits}</td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            )}
        </>
    );
};
```

CourseOverviewTable component

```tsx
import { useState } from 'react';
import { Lecturer } from '@types';
import LecturersOverviewTable from '@components/lecturers/LecturerOverviewTable';
import CourseOverviewTable from '@components/courses/CourseOverviewTable';

type Props = {
  initialLecturers: Lecturer[];
};

const LecturersClientPage = ({ initialLecturers }: Props) => {
  const [lecturers] = useState<Lecturer[]>(initialLecturers);
  const [selectedLecturer, setSelectedLecturer] = useState<Lecturer | null>(
    null
  );

  return (
    <>
      <section>
        <LecturersOverviewTable
          lecturers={lecturers}
          selectLecturer={setSelectedLecturer}
        />
      </section>

      {selectedLecturer && (
        <section className="mt-5">
          <h2 className="text-center">
            Courses taught by {selectedLecturer.user.firstName}
          </h2>
          {selectedLecturer.courses && (
            <CourseOverviewTable lecturer={selectedLecturer} />
          )}
        </section>
      )}
    </>
  );
};

export default LecturersClientPage;
```

A state variable to “remember” that a lecturer was clicked in the table.

The state setter function is passed as a callback, so the table component can immediately change the state of this component. This state change will trigger a re-render of this component causing the CourseOverviewTable to be displayed.